Documentação do Projeto PetHouse

Projeto Integrador — Modelagem de Dados

1 INTRODUÇÃO

Este documento apresenta a documentação completa do projeto Pet House. Na primeira entrega trouxemos o DER sem a análise do negócio por trás dele. O professor apontou isso na devolutiva: o DER precisa nascer da análise, e cada decisão de modelagem precisa ter uma razão ligada ao funcionamento real da empresa.

Por isso, organizamos a documentação na ordem que faz sentido: primeiro explicamos a empresa, levantamos os problemas, definimos o que o sistema precisa fazer, desenhamos os fluxos e só depois explicamos o DER. Nesta segunda entrega também incluímos o modelo lógico, a normalização corrigida, consultas SQL completas, indicadores de BI e possibilidades de IA, além de todas as correções que o professor indicou.

1.1 Objetivo geral

Documentar a modelagem de dados de um sistema para o pet shop, usando um banco de dados relacional MySQL, de forma a substituir os controles manuais que a empresa usa hoje.

1.2 Objetivos específicos

- Descrever o funcionamento atual do pet shop e suas dificuldades;
- Relacionar cada problema a uma necessidade e a uma parte do modelo de dados;
- Formalizar requisitos funcionais, não funcionais e regras de negócio;
- Representar os processos principais em fluxogramas;
- Explicar as entidades, os relacionamentos, as cardinalidades e os atributos do DER;
- Apresentar o modelo lógico derivado do DER;
- Demonstrar a normalização das tabelas até a 3FN com as correções aplicadas;
- Fornecer script SQL completo com CREATE, INSERT, SELECT, JOIN, UPDATE e DELETE;
- Apresentar consultas gerenciais, de auditoria e indicadores de BI;
- Discutir possibilidades de uso de Inteligência Artificial nos dados do sistema.

1A REVISÃO DA 1ª ENTREGA

Esta seção registra o que foi apontado na devolutiva do professor e como cada ponto foi endereçado nesta segunda entrega.

Ponto levantado	Como era (1ª entrega)	Como ficou (2ª entrega)	Por quê
Falta análise do negócio	Apenas o DER sem contexto	Seções 2, 3, 4 e 5 completas com problemas, necessidades, requisitos e regras	O DER deve ser consequência da análise
Orçamento criado antes do Pedido	Relacionamento "gera" com PEDIDO → ORCAMENTO, sentido invertido na prática	Mantido no DER; explicado na seção 8.3 e corrigido no SQL com pedido_id opcional em ORCAMENTO	A sequência real é: orçamento primeiro, pedido depois

FEEDBACK fora da 3FN	cliente_id e pedido_id na tabela FEEDBACK (cliente redundante)	Script SQL corrigido: cliente_id removido de FEEDBACK; cliente obtido via JOIN com PEDIDO	Todo pedido já tem cliente; coluna redundante viola a 3FN
Falta modelo lógico	Apenas DER conceitual	Seção 8A com tabelas relacionais e chaves explícitas	Conversão DER→lógico é etapa obrigatória
Falta classificação de entidades	Sem mencionar entidades fracas ou associativas	Seção 8B com classificação explícita de cada entidade	Diferença entre entidade forte, fraca e associativa é conteúdo da disciplina
Script SQL incompleto	Apenas CREATE e INSERT (pg_dump)	Seção 11 ampliada com SELECT, JOIN, UPDATE, DELETE, consultas gerenciais e auditoria	O professor exige operações CRUD completas
Ausência de consultas gerenciais	Nenhuma consulta de relatório	Seção 11.5 com 10 consultas gerenciais e 5 de auditoria	Demonstra uso prático do banco
Ausência de BI e IA	Palavras não apareciam no documento	Seção 13A com indicadores de BI e possibilidades de IA	Exigência da 2ª entrega
README incompleto	2 de 16 seções preenchidas	Seção 14 com README completo pronto para o GitHub	Documentação de repositório é obrigatória

2 CARACTERIZAÇÃO DA EMPRESA

A Pet House é um pet shop de pequeno porte que combina a venda de produtos (rações, acessórios, produtos de higiene e outros itens para animais) com a prestação de serviços agendados (banho, tosa e consultas de rotina). O atendimento ocorre no balcão, por telefone e por WhatsApp. Os produtos são adquiridos de fornecedores e mantidos em estoque até a venda.

2.1 Setores e processos

Setor	Serviços e processos
Atendimento	Cadastro de clientes e pets, agendamento de procedimentos, emissão de orçamentos e pesquisa de satisfação.
Serviços	Execução dos procedimentos agendados para cada pet.
Vendas	Registro de pedidos com seus itens e recebimento do pagamento.
Compras e estoque	Escolha de fornecedores, compra e recebimento de mercadorias e controle das quantidades em estoque.
Financeiro	Acompanhamento de valores recebidos e pagos, fluxo de caixa e conferência com o extrato bancário.

2.2 Funcionamento atual

Todos os processos acima são realizados de forma manual. Os dados de clientes e pets ficam em cadernos ou planilhas, a agenda é anotada em papel ou em conversas de WhatsApp, os orçamentos são feitos de maneira informal, o estoque é conferido visualmente e as compras são combinadas por telefone, sem registro dos preços praticados por cada fornecedor. O controle financeiro é feito em caderno de caixa ou planilha, sem conferência sistemática com o banco.

2.3 Dificuldades atuais

- Informações duplicadas, incompletas ou perdidas, o que impede conhecer o histórico do cliente e do pet;
- Conflito de horários e esquecimento de agendamentos;
- Diferença entre o preço combinado no orçamento e o preço cobrado na venda;
- Venda de produtos sem estoque e falta de reposição no momento certo;
- Desconhecimento de qual fornecedor oferece o melhor preço;
- Falta de visão sobre o dinheiro que entra e sai e sobre os valores pendentes;
- Ausência de registro da opinião dos clientes.

3 PROBLEMAS E NECESSIDADES

A tabela relaciona cada problema concreto à necessidade que ele gera e à parte do DER que a atende.

Nº	Problema	Necessidade	Entidades do DER
P1	Cadastro disperso e duplicado de clientes e pets	Cadastro único, com cada pet ligado ao seu tutor	CLIENTE, PET
P2	Agenda em papel e WhatsApp, sem controle de situação	Agendamento registrado com data, horário, procedimento e status	AGENDAMENTO
P3	Orçamentos e vendas sem registro padronizado	Registro formal de orçamentos e pedidos, com itens, preços e totais	ORCAMENTO, PEDIDO, ITEM_PEDIDO
P4	Catálogo de produtos desorganizado	Produtos descritos, com preço e classificados em categorias	PRODUTO, CATEGORIA
P5	Estoque conferido a olho	Quantidade atual e mínima por produto, com alerta de reposição	ESTOQUE
P6	Compras sem registro de fornecedor e preço	Cadastro de fornecedores e do preço de compra de cada produto fornecido	FORNECEDOR, FORNECEDOR_PRODUTO
P7	Financeiro em caderno, sem conferência bancária	Acompanhar valores e situação dos pedidos, fluxo de caixa e conciliação	PEDIDO (total e status)
P8	Opinião do cliente não registrada	Registrar nota e comentário do cliente	FEEDBACK

4 SOLUÇÃO PROPOSTA

A solução é um ERP (Enterprise Resource Planning) integrado a um banco de dados MySQL. O ERP centraliza em um único sistema os módulos de cadastros, agendamento, orçamentos e vendas, compras e fornecedores, estoque e acompanhamento financeiro. O banco de dados armazena o modelo representado no DER e impede dados inconsistentes por meio de chaves primárias, chaves estrangeiras e restrições.

Para esta entrega, o professor pediu que o banco fosse implementado em MySQL, usando o phpMyAdmin como ferramenta de importação. O MySQL é um sistema de banco de dados gratuito, muito usado no mercado, com suporte a transações, chaves estrangeiras e restrições CHECK que garantem que os dados fiquem sempre consistentes.

Com o sistema, os processos que eram manuais passam a ser registrados no banco de dados no momento em que acontecem, e as consultas (histórico do cliente, agenda do dia, estoque baixo, vendas por período) deixam de depender de anotações dispersas.

5 REQUISITOS

5.1 Requisitos funcionais

ID	Requisito	Entidade
RF01	O sistema deve cadastrar, consultar, alterar e excluir clientes (nome, telefone, e-mail e endereço).	CLIENTE
RF02	O sistema deve cadastrar pets (nome, tipo, raça e idade) vinculados a um cliente.	PET
RF03	O sistema deve registrar agendamentos de um pet, informando data, horário e tipo de procedimento.	AGENDAMENTO
RF04	O sistema deve permitir alterar o status do agendamento (agendado, concluído, cancelado, não compareceu).	AGENDAMENTO
RF05	O sistema deve consultar a agenda por data e por pet.	AGENDAMENTO
RF06	O sistema deve cadastrar categorias e produtos (nome, descrição, preço e categoria).	CATEGORIA, PRODUTO
RF07	O sistema deve registrar orçamentos de clientes com data, status e total.	ORCAMENTO
RF08	O sistema deve registrar pedidos de clientes com data, status e total.	PEDIDO
RF09	O sistema deve registrar os itens de cada pedido, com produto, quantidade e preço unitário.	ITEM_PEDIDO
RF10	O sistema deve calcular o total do pedido a partir de seus itens.	PEDIDO, ITEM_PEDIDO
RF11	O sistema deve manter a quantidade atual e o estoque mínimo de cada produto.	ESTOQUE
RF12	O sistema deve atualizar o estoque após a venda de produtos e após o recebimento de mercadorias.	ESTOQUE
RF13	O sistema deve alertar quando a quantidade de um produto for menor que o estoque mínimo.	ESTOQUE
RF14	O sistema deve cadastrar fornecedores (nome, CNPJ, telefone, e-mail e endereço).	FORNECEDOR
RF15	O sistema deve registrar os produtos fornecidos por cada fornecedor, com preço de compra e data de fornecimento.	FORNECEDOR_PRODUTO
RF16	O sistema deve registrar o feedback do cliente sobre um pedido (nota e comentário).	FEEDBACK
RF17	O sistema deve registrar o status do pedido para acompanhar o que foi pago e o que está pendente.	PEDIDO
RF18	O sistema deve emitir relatórios de vendas por período, estoque baixo, agenda do dia e histórico do cliente.	Todas

5.2 Requisitos não funcionais

ID	Requisito
RNF01	Os dados devem ser armazenados em banco de dados MySQL, importado via phpMyAdmin.
RNF02	O acesso ao sistema deve exigir login e senha, com perfis de acesso distintos (atendente, vendedor e gestor).
RNF03	As senhas devem ser armazenadas de forma criptografada.
RNF04	O tratamento dos dados pessoais dos clientes deve respeitar a LGPD.
RNF05	As consultas mais comuns (busca de cliente, produto e agenda do dia) devem responder em até 3 segundos.
RNF06	Deve haver cópia de segurança (backup) diária do banco de dados.
RNF07	A interface deve ser utilizável em computador e em celular.
RNF08	As operações que alteram mais de uma tabela devem ser transacionais (concluídas por inteiro ou desfeitas).

6 REGRAS DE NEGÓCIO

ID	Regra
RN01	Todo pet pertence a um único cliente; um cliente pode ter vários pets.
RN02	Todo agendamento é de um único pet; um pet pode ter vários agendamentos.
RN03	O status do agendamento deve ser um dos valores previstos: Agendado, Concluído, Cancelado ou Não compareceu.
RN04	Todo pedido pertence a um único cliente; um cliente pode realizar vários pedidos.
RN05	Todo pedido possui ao menos um item, e todo item pertence a um único pedido.
RN06	Cada item de pedido refere-se a um único produto; um produto pode estar em vários itens.
RN07	O preço unitário do item é gravado no momento da venda e não muda se o preço do produto for alterado depois.
RN08	O total do pedido é a soma de quantidade × preço unitário de todos os seus itens.
RN09	Todo produto pertence a uma única categoria; uma categoria pode agrupar vários produtos.
RN10	Cada produto possui exatamente um registro de estoque, e a quantidade nunca pode ser negativa.
RN11	Não é permitido vender uma quantidade maior que o saldo em estoque.
RN12	Quando a quantidade em estoque ficar menor que o estoque mínimo, o sistema deve emitir alerta de reposição.
RN13	Um fornecedor pode fornecer vários produtos e um produto pode ser fornecido por vários fornecedores; cada combinação tem preço de compra e data de fornecimento próprios.
RN14	O CNPJ de fornecedor não pode se repetir.
RN15	Um orçamento pertence a um cliente e possui status; um orçamento recusado não pode virar pedido. O orçamento é criado antes do pedido; quando aprovado, gera um pedido.
RN16	A nota do feedback deve estar entre 1 e 5, e todo feedback é de um cliente e se refere a um pedido. O cliente do feedback é obtido via JOIN com PEDIDO (não armazenado redundantemente).

7 FLUXOGRAMAS DOS PROCESSOS

Os fluxogramas a seguir representam os processos do pet shop. Legenda: retângulo de cantos arredondados = início ou fim; retângulo = atividade; losango = decisão; cilindro = gravação de dados no banco de dados.

7.1 Agendamento e atendimento

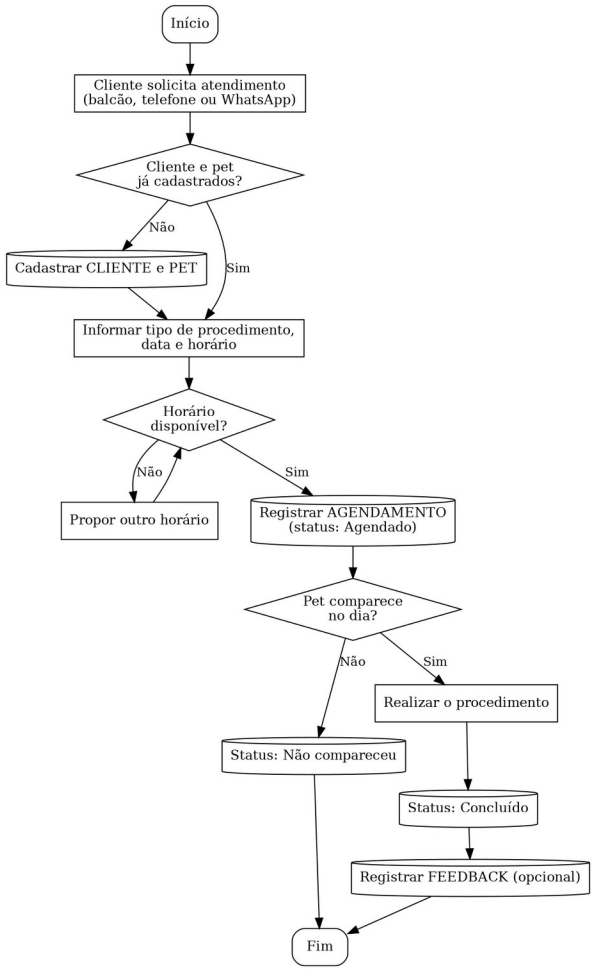


Figura 1 – Fluxograma de agendamento e atendimento

7.2 Orçamento e venda

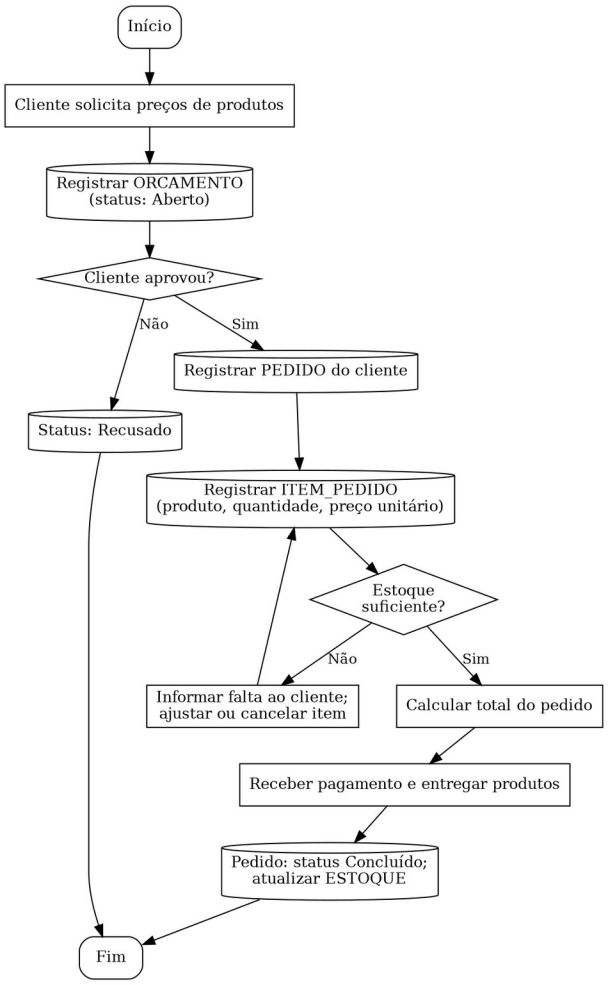

Início
Cliente solicita preços de produtos
Registrar ORCAMENTO
(status: Aberto)
Cliente aprovou?
Não
Sim
Registrar PEDIDO do cliente
Status: Recusado
Registrar ITEM_PEDIDO
(produto, quantidade, preço unitário)
Estoque
suficiente?
Não
Sim
Informar falta ao cliente;
ajustar ou cancelar item
Calcular total do pedido
Receber pagamento e entregar produtos
Pedido: status Concluído;
atualizar ESTOQUE
Fim

Figura 2 – Fluxograma de orçamento e venda

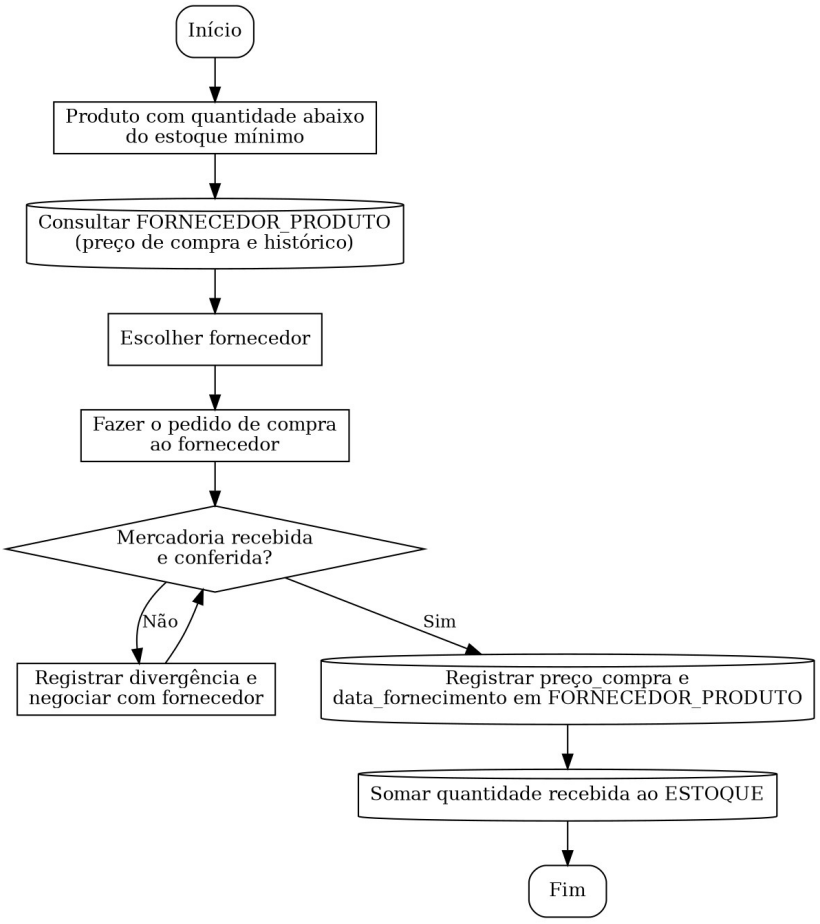


Figura 3 – Fluxograma de compras e reposição de mercadorias

7.4 Controle de estoque

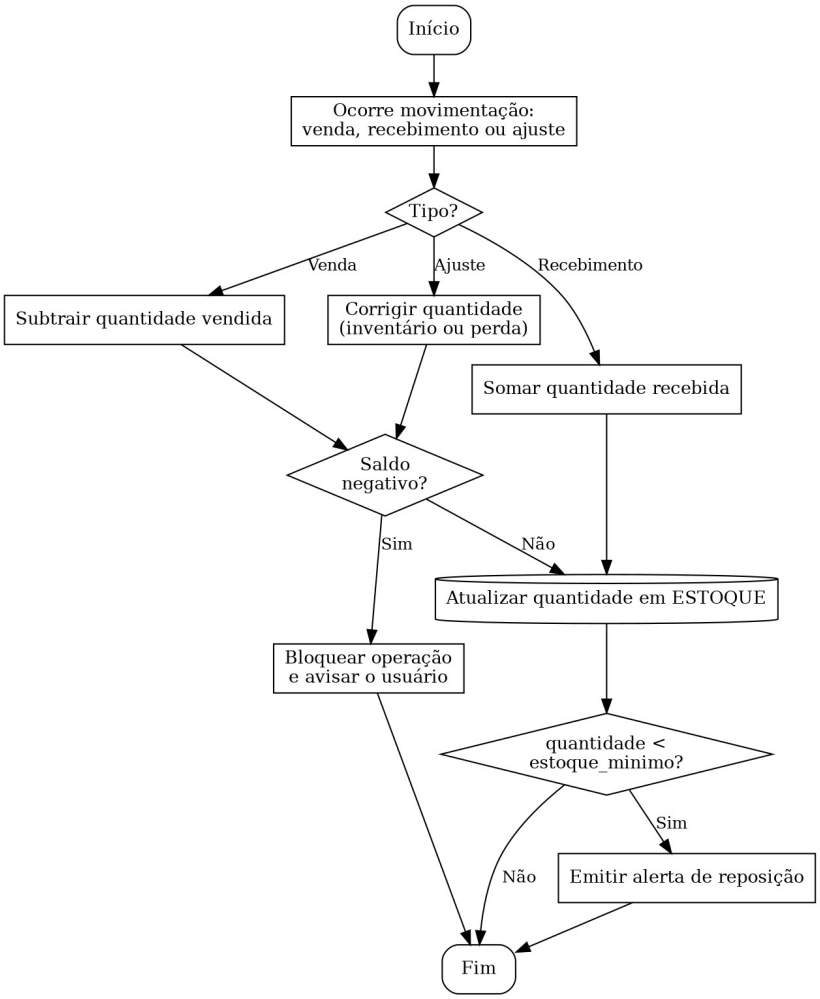


Figura 4 – Fluxograma de controle de estoque

7.5 Gestão financeira

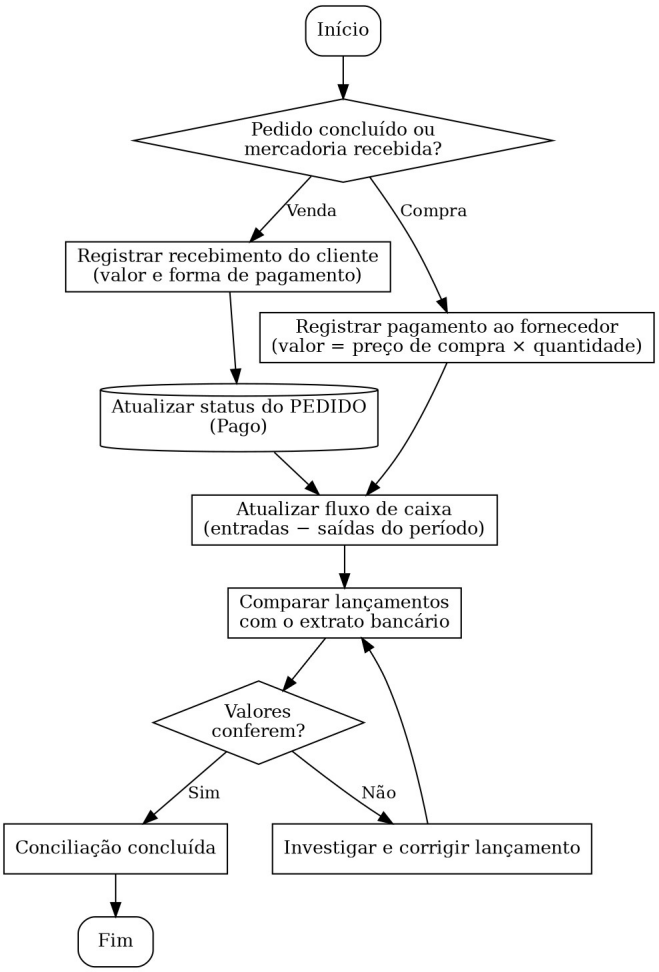


Figura 5 – Fluxograma da gestão financeira

8 MODELO DE DADOS (DER)

A Figura 6 apresenta o modelo relacional do projeto, derivado do DER e correspondente às 12 tabelas implementadas no script MySQL. Cada retângulo representa uma tabela e apresenta sua chave primária (PK), chaves estrangeiras (FK) e, quando necessário, restrições de unicidade. As setas indicam os relacionamentos e suas cardinalidades. A tabela FORNECEDOR_PRODUTO, destacada com borda tracejada, resolve o relacionamento muitos-para-muitos entre PRODUTO e FORNECEDOR.

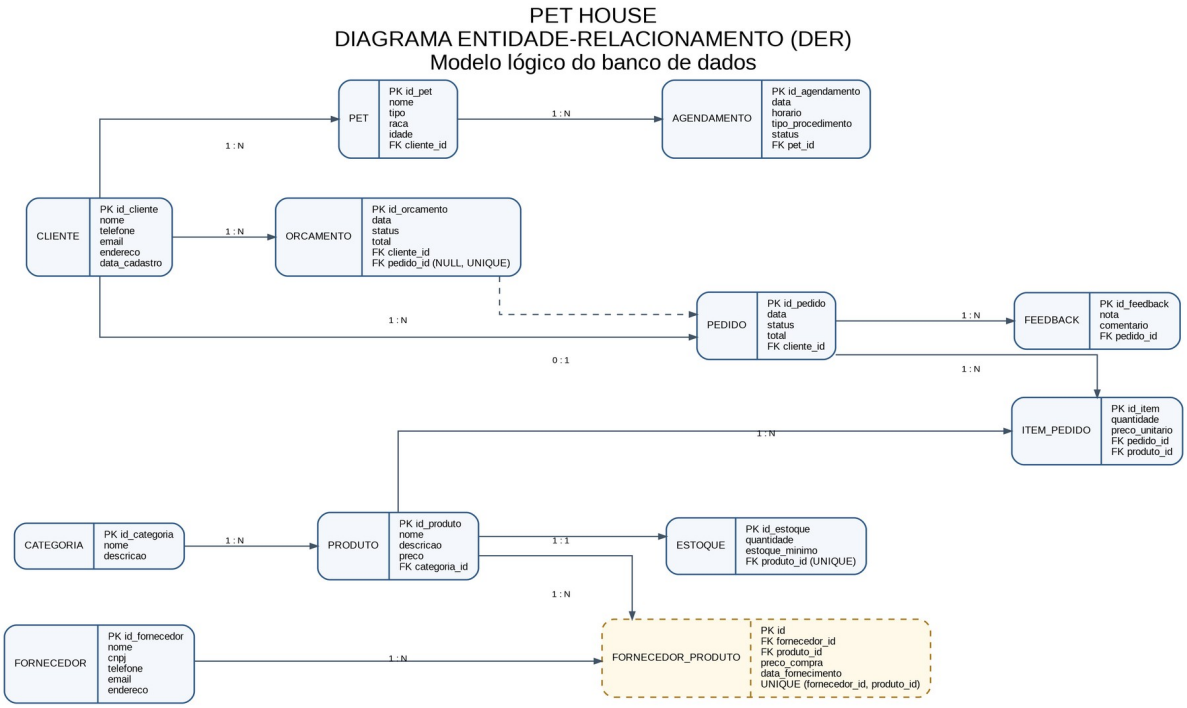


Figura 6 – Diagrama Entidade-Relacionamento (DER) do sistema Pet House

8.1 Entidades e necessidade atendida

Entidade	Descrição	Necessidade
CLIENTE	Pessoa que compra produtos ou leva pets para atendimento	P1
PET	Animal pertencente a um cliente	P1
AGENDAMENTO	Reserva de data e horário de um procedimento para um pet	P2
ORCAMENTO	Proposta de valor feita a um cliente antes da venda	P3
PEDIDO	Venda realizada a um cliente	P3, P7
ITEM_PEDIDO	Produto, quantidade e preço de cada linha do pedido; resolve o relacionamento entre pedido e produto	P3
CATEGORIA	Grupo ao qual os produtos pertencem	P4
PRODUTO	Item vendido pela loja	P3, P4
ESTOQUE	Quantidade atual e mínima de um produto	P5

FORNECEDOR	Empresa que abastece a loja	P6
FORNECEDOR_PRODUTO	Preço de compra e data de fornecimento de cada produto por fornecedor	P6
FEEDBACK	Nota e comentário do cliente sobre um pedido	P8

8.2 Classificação das entidades

Cada entidade do modelo é classificada em forte, fraca ou associativa, conforme os critérios da modelagem conceitual.

Entidade	Classificação	Justificativa
CLIENTE	Forte	Existe independentemente de qualquer outra entidade; tem chave própria (id_cliente).
PET	Fraca (parcialmente)	Depende de CLIENTE para existir — um pet sem tutor não tem sentido no negócio; a chave id_pet é gerada pelo sistema, mas a existência depende de cliente_id.
AGENDAMENTO	Fraca	Depende de PET para existir; um agendamento sem animal não tem sentido.
ORCAMENTO	Forte	Tem identidade própria (id_orcamento) e pode existir antes de qualquer pedido (pedido_id é opcional).
PEDIDO	Forte	Tem identidade própria (id_pedido); depende de CLIENTE apenas por regra de negócio, não existencialmente.
ITEM_PEDIDO	Associativa + fraca	Resolve o relacionamento N:N entre PEDIDO e PRODUTO; sem o pedido ou o produto não existe.
CATEGORIA	Forte	Existe independentemente dos produtos.
PRODUTO	Forte	Tem identidade própria; depende de CATEGORIA apenas por classificação.
ESTOQUE	Fraca	Depende de PRODUTO para existir; é uma extensão do cadastro do produto.
FORNECEDOR	Forte	Existe independentemente dos produtos que fornece.
FORNECEDOR_PRODUTO	Associativa	Resolve o relacionamento N:N entre FORNECEDOR e PRODUTO; sem os dois não existe.
FEEDBACK	Fraca	Depende de PEDIDO para existir; uma avaliação sem pedido não tem sentido.

8.3 Relacionamentos e cardinalidades

Cada cardinalidade é apresentada nos dois sentidos, com o mínimo e o máximo de ocorrências, e justificada por uma regra de negócio.

Relacionamento	Sentido A → B	Sentido B → A	Justificativa
CLIENTE tem PET (1:N)	Um cliente tem de 0 a N pets (0,N)	Um pet pertence a 1 cliente (1,1)	Um cliente pode apenas comprar produtos; todo pet tem um tutor (RN01).
PET possui AGENDAMENTO (1:N)	Um pet tem de 0 a N agendamentos (0,N)	Um agendamento é de 1 pet (1,1)	Pet recém-cadastrado ainda não tem agenda; cada reserva é de um animal (RN02).
CLIENTE realiza PEDIDO (1:N)	Um cliente faz de 0 a N pedidos (0,N)	Um pedido é de 1 cliente (1,1)	Cliente novo ainda não comprou; toda venda tem comprador (RN04).
PEDIDO contém ITEM_PEDIDO (1:N)	Um pedido tem de 1 a N itens (1,N)	Um item pertence a 1 pedido (1,1)	Pedido sem item não existe (RN05).
ITEM_PEDIDO refere-se a PRODUTO (N:1)	Um item refere-se a 1 produto (1,1)	Um produto está em 0 a N itens (0,N)	Produto novo pode não ter sido vendido (RN06).
PRODUTO pertence a CATEGORIA (N:1)	Um produto tem 1 categoria (1,1)	Uma categoria tem de 0 a N produtos (0,N)	Categoria pode ser criada antes dos produtos; todo produto é classificado (RN09).
PRODUTO possui ESTOQUE (1:1)	Um produto tem 1 estoque (1,1)	Um estoque é de 1 produto (1,1)	Saldo único por produto, evitando contagens divergentes (RN10).
PRODUTO ↔ FORNECEDOR via FORNECEDOR_PRODUTO (N:N)	Um produto tem de 0 a N fornecedores (0,N)	Um fornecedor fornece de 0 a N produtos (0,N)	Produto pode ter vários fornecedores, cada um com preço e data próprios (RN13).
CLIENTE emite ORCAMENTO (1:N)	Um cliente emite de 0 a N orçamentos (0,N)	Um orçamento pertence a 1 cliente (1,1)	Orçamento sem cliente não tem sentido (RN15).
ORCAMENTO gera PEDIDO (0:1)	Um orçamento gera de 0 a 1 pedido (0,1)	Um pedido pode vir de 0 ou 1 orçamento (0,1)	O orçamento é criado primeiro; quando aprovado, origina um pedido (RN15).
PEDIDO recebe FEEDBACK (1:N)	Um pedido recebe de 0 a N feedbacks (0,N)	Um feedback refere-se a 1 pedido (1,1)	Nem todo pedido é avaliado; toda avaliação é sobre um pedido (RN16).

8.4 Notas de consistência apontadas na devolutiva

Orçamento e pedidos. No DER, o orçamento é criado antes do pedido (fluxograma 7.2). O script SQL corrigido inclui o campo pedido_id como opcional (NULL) em ORCAMENTO, permitindo registrar orçamentos ainda abertos. Quando aprovado, o pedido_id é preenchido. Isso materializa o vínculo sem exigir que o pedido exista no momento do orçamento.

Fornecedores e estoque. O relacionamento entre FORNECEDOR e PRODUTO é representado pela tabela associativa FORNECEDOR_PRODUTO, que registra o preço de compra e a data de fornecimento. A tabela ESTOQUE mantém somente a quantidade atual, o estoque mínimo e a referência ao produto; não armazena fornecedor_id.

FEEDBACK sem cliente_id. Conforme apontado na 3FN (seção 10), a coluna cliente_id não faz parte de FEEDBACK no script SQL atualizado. O cliente é identificado pelo relacionamento FEEDBACK → PEDIDO → CLIENTE, evitando armazenar novamente uma informação que já pode ser obtida pelo pedido.

8.5 Escopo do DER em relação aos demais itens

Serviços e agendamentos: o DER representa o serviço pelo atributo tipo_procedimento da entidade AGENDAMENTO. O cadastro de funcionários responsáveis, de serviços com preço próprio e das compras e movimentações de estoque como registros separados não faz parte da versão atual do modelo e é indicado como evolução. Gestão financeira: nesta versão, o acompanhamento financeiro se apoia nos atributos total e status de PEDIDO e ORCAMENTO; contas a pagar, contas a receber e movimentos de caixa são tratados como processo (fluxograma 7.5) e indicados como evolução do banco de dados.

8A MODELO LÓGICO (DER → RELACIONAL)

O modelo lógico é obtido pela conversão do DER conceitual para tabelas relacionais, explicitando todas as chaves primárias (PK), chaves estrangeiras (FK) e restrições (UNIQUE, NOT NULL, CHECK). Os tipos de dados seguem o MySQL (INT, VARCHAR, DECIMAL, TEXT, DATE, TIME, DATETIME).

Notação: PK = chave primária | FK = chave estrangeira | U = unique | NN = not null | CK = check

Tabela	**Atributos e restrições**
CLIENTE	id_cliente PK, nome NN, telefone, email U, endereco, data_cadastro NN DEFAULT CURRENT_TIMESTAMP
PET	id_pet PK, nome NN, tipo NN, raca, idade CK(>=0), cliente_id FK→CLIENTE NN
AGENDAMENTO	id_agendamento PK, data NN, horario NN, tipo_procedimento NN, status NN CK(IN agendado/concluido/cancelado/nao_compareceu), pet_id FK→PET NN
CATEGORIA	id_categoria PK, nome NN U, descricao
PRODUTO	id_produto PK, nome NN, descricao, preco NN CK(>0), categoria_id FK→CATEGORIA NN
ESTOQUE	id_estoque PK, quantidade NN CK(>=0), estoque_minimo NN CK(>=0), produto_id FK→PRODUTO NN U
FORNECEDOR	id_fornecedor PK, nome NN, cnpj NN U, telefone, email, endereco
FORNECEDOR_PRODUTO	id PK, fornecedor_id FK→FORNECEDOR NN, produto_id FK→PRODUTO NN, preco_compra NN CK(>0), data_fornecimento NN | UNIQUE(fornecedor_id, produto_id)
PEDIDO	id_pedido PK, data NN DEFAULT CURRENT_TIMESTAMP, status NN, total NN CK(>=0), cliente_id FK→CLIENTE NN
ORCAMENTO	id_orcamento PK, data NN DEFAULT CURRENT_TIMESTAMP, status NN CK(IN aberto/aprovado/recusado), total NN CK(>=0), cliente_id FK→CLIENTE NN, pedido_id FK→PEDIDO (NULL permitido, UNIQUE)
ITEM_PEDIDO	id_item PK, quantidade NN CK(>0), preco_unitario NN CK(>0), pedido_id FK→PEDIDO NN, produto_id FK→PRODUTO NN
FEEDBACK	id_feedback PK, nota NN CK(1..5), comentario, pedido_id FK→PEDIDO NN | (cliente obtido via JOIN PEDIDO→CLIENTE)

9 DICIONÁRIO DE DADOS

Legenda: Obrig. = obrigatório (NOT NULL); PK = chave primária; FK = chave estrangeira; derivado = valor calculado a partir de outros dados. Os tipos usados são os do MySQL.

9.1 CLIENTE

Atributo	Tipo	Obrig.	Descrição e regras
id_cliente	INT (AUTO_INCREMENT)	Sim	PK. Identificador do cliente.
nome	VARCHAR(120)	Sim	Nome completo do cliente.
telefone	VARCHAR(20)	Não	Telefone ou WhatsApp para contato.
email	VARCHAR(150)	Não	E-mail para envio de orçamentos. UNIQUE.
endereco	VARCHAR(255)	Não	Endereço do cliente.
data_cadastro	DATETIME	Sim	Data e hora do cadastro. DEFAULT CURRENT_TIMESTAMP.

9.2 PET

Atributo	Tipo	Obrig.	Descrição e regras
id_pet	INT (AUTO_INCREMENT)	Sim	PK.
nome	VARCHAR(100)	Sim	Nome do animal.
tipo	VARCHAR(50)	Sim	Espécie do animal (cão, gato etc.).
raca	VARCHAR(100)	Não	Raça, quando conhecida.
idade	SMALLINT	Não	Idade em anos. CHECK >= 0.
cliente_id	INTEGER	Sim	FK para CLIENTE. Tutor do pet (RN01).

9.3 AGENDAMENTO

Atributo	Tipo	Obrig.	Descrição e regras
id_agendamento	INT (AUTO_INCREMENT)	Sim	PK.
data	DATE	Sim	Dia do atendimento.
horario	TIME	Sim	Hora do atendimento.
tipo_procedimento	VARCHAR(50)	Sim	Banho, tosa, consulta etc.
status	VARCHAR(20)	Sim	Agendado, Concluído, Cancelado ou Não

			compareceu. CHECK. Padrão: Agendado (RN03).
pet_id	INTEGER	Sim	FK para PET (RN02).

9.4 ORCAMENTO

Atributo	Tipo	Obrig.	Descrição e regras
id_orcamento	INT (AUTO_INCREMENT)	Sim	PK.
data	DATETIME	Sim	Data de emissão. DEFAULT CURRENT_TIMESTAMP.
status	VARCHAR(20)	Sim	Aberto, Aprovado ou Recusado. CHECK. Padrão: Aberto (RN15).
total	DECIMAL(10,2)	Sim	Valor total proposto. CHECK >= 0.
cliente_id	INTEGER	Sim	FK para CLIENTE.
pedido_id	INTEGER	Não	FK para PEDIDO. NULL enquanto orçamento estiver aberto ou recusado (RN15).

9.5 PEDIDO

Atributo	Tipo	Obrig.	Descrição e regras
id_pedido	INT (AUTO_INCREMENT)	Sim	PK.
data	DATETIME	Sim	Data e hora da venda. DEFAULT CURRENT_TIMESTAMP.
status	VARCHAR(20)	Sim	Pendente, Pago, Entregue ou Cancelado. CHECK.
total	DECIMAL(10,2)	Sim	Derivado: soma de quantidade × preço_unitário dos itens (RN08). CHECK >= 0.
cliente_id	INTEGER	Sim	FK para CLIENTE (RN04).

9.6 ITEM_PEDIDO

Atributo	Tipo	Obrig.	Descrição e regras
id_item	INT (AUTO_INCREMENT)	Sim	PK.
quantidade	INTEGER	Sim	CHECK > 0 e <= saldo em estoque (RN11).

preco_unitario	DECIMAL(10,2)	Sim	Preço do produto no momento da venda. CHECK > 0 (RN07).
pedido_id	INTEGER	Sim	FK para PEDIDO (RN05).
produto_id	INTEGER	Sim	FK para PRODUTO (RN06).

9.7 CATEGORIA

Atributo	Tipo	Obrig.	Descrição e regras
id_categoria	INT (AUTO_INCREMENT)	Sim	PK.
nome	VARCHAR(100)	Sim	Nome da categoria. UNIQUE.
descricao	TEXT	Não	Descrição da categoria.

9.8 PRODUTO

Atributo	Tipo	Obrig.	Descrição e regras
id_produto	INT (AUTO_INCREMENT)	Sim	PK.
nome	VARCHAR(100)	Sim	Nome comercial.
descricao	VARCHAR(200)	Não	Marca, tamanho e detalhes.
preco	DECIMAL(10,2)	Sim	Preço de venda atual. CHECK > 0.
categoria_id	INTEGER	Sim	FK para CATEGORIA (RN09).

9.9 ESTOQUE

Atributo	Tipo	Obrig.	Descrição e regras
id_estoque	INT (AUTO_INCREMENT)	Sim	PK.
quantidade	INTEGER	Sim	Saldo atual do produto. CHECK >= 0 (RN10).
estoque_minimo	INTEGER	Sim	Nível que dispara o alerta de reposição. CHECK >= 0 (RN12).
produto_id	INTEGER	Sim	FK para PRODUTO. UNIQUE: um estoque por produto.

9.10 FORNECEDOR

Atributo	Tipo	Obrig.	Descrição e regras
id_fornecedor	INT (AUTO_INCREMENT)	Sim	PK.
nome	VARCHAR(150)	Sim	Razão social ou nome do fornecedor.
cnpj	VARCHAR(18)	Sim	UNIQUE (RN14).
telefone	VARCHAR(20)	Não	Contato comercial.
email	VARCHAR(150)	Não	E-mail para pedidos de compra.
endereco	VARCHAR(255)	Não	Endereço do fornecedor.

9.11 FORNECEDOR_PRODUTO

Atributo	Tipo	Obrig.	Descrição e regras
id	INT (AUTO_INCREMENT)	Sim	PK.
fornecedor_id	INTEGER	Sim	FK para FORNECEDOR.
produto_id	INTEGER	Sim	FK para PRODUTO. UNIQUE(fornecedor_id, produto_id) (RN13).
preco_compra	DECIMAL(10,2)	Sim	Preço cobrado pelo fornecedor. CHECK > 0.
data_fornecimento	DATE	Sim	Data do último fornecimento.

9.12 FEEDBACK

Atributo	Tipo	Obrig.	Descrição e regras
id_feedback	INT (AUTO_INCREMENT)	Sim	PK.
nota	SMALLINT	Sim	De 1 a 5. CHECK (RN16).
comentario	VARCHAR(300)	Não	Texto livre do cliente.
pedido_id	INTEGER	Sim	FK para PEDIDO (RN16). O cliente é obtido via JOIN (cliente_id removido da tabela para atender à

			3FN).

10 NORMALIZAÇÃO

Normalizar é organizar as tabelas para que cada informação fique guardada em um único lugar, sem repetição, reduzindo anomalias de inserção, atualização e exclusão.

10.1 Dois conceitos importantes

Dependência funcional: ao conhecer um dado, descobrimos outro. Por exemplo, sabendo o id_produto, descobrimos o nome e o preço do produto. Escrevemos assim: id_produto → nome, preco.

Chaves do nosso banco: todas as tabelas têm como chave primária um número automático (id). Na 2FN, as tabelas com chave composta possível são FORNECEDOR_PRODUTO e ITEM_PEDIDO. Na 3FN, olhamos também as colunas UNIQUE como e-mail do cliente, nome da categoria e CNPJ do fornecedor.

10.2 Primeira Forma Normal (1FN)

Cada campo guarda um único valor; não há colunas repetidas nem grupos repetitivos.

Situação	Sem separar	Problema	O que fizemos
Produtos de um pedido	Colunas produto1, qtd1, produto2, qtd2... no pedido	O pedido teria limite de itens e não daria para somar vendas	Criamos ITEM_PEDIDO: cada linha é um produto de um pedido, com quantidade e preço
Fornecedores de um produto	Uma coluna com vários fornecedores dentro de PRODUTO	Não haveria onde guardar o preço de compra e a data de cada fornecedor	Criamos FORNECEDOR_PRODUTO: cada linha liga um fornecedor a um produto, com preço e data (RN13)
Serviços do agendamento	tipo_procedimento = "banho, tosa"	Dois serviços no mesmo campo, mas com um só status	tipo_procedimento guarda um único serviço. Banho e tosa do mesmo pet viram dois agendamentos
Telefone do cliente	telefone1, telefone2, telefone3	Colunas repetidas para a mesma informação	A coluna telefone guarda um único contato
Endereço	Rua, número, bairro e cidade em um campo só	Poderia ser dividido	Mantivemos como texto único; se precisarmos buscar por bairro, dividimos futuramente

Resultado: todas as colunas guardam um só valor, não há colunas repetidas e todas as tabelas têm chave primária. O banco está na 1FN.

10.3 Segunda Forma Normal (2FN)

Cada informação depende da chave inteira, não de um pedaço dela. Aplica-se às tabelas com chave composta possível.

Tabela	Quem depende de	Conferência	Resultado

	quem		
FORNECEDOR_PRODUTO (fornecedor_id + produto_id)	fornecedor + produto → preco_compra, data_fornecimento	O preço depende dos dois juntos: o mesmo fornecedor cobra preços diferentes por produto e o mesmo produto tem preços diferentes por fornecedor (RN13)	2FN atendida
ITEM_PEDIDO (pedido_id + produto_id)	pedido + produto → quantidade, preco_unitario	O preco_unitario é o preço cobrado naquela venda (RN07), dependendo do pedido e do produto juntos; nome e preço atual ficam em PRODUTO	2FN atendida

Resultado: nenhuma informação depende só de um pedaço da chave. O banco está na 2FN.

10.4 Terceira Forma Normal (3FN)

Cada informação depende só da chave primária, não de outra coluna não-chave.

Situação	Quem define quem	Problema	O que fizemos
PRODUTO e CATEGORIA	id_produto → categoria_id; categoria_id → nome e descrição	Nome da categoria repetido em cada produto	Resolvido: dados da categoria ficam em CATEGORIA; PRODUTO guarda só o categoria_id
AGENDAMENTO, PET e CLIENTE	id_agendamento → pet_id → cliente_id → dados do cliente	Dados do cliente repetidos em AGENDAMENTO e PET	Resolvido: AGENDAMENTO guarda só pet_id; PET e PEDIDO guardam só cliente_id. Para ver os dados, usamos JOIN
FEEDBACK: cliente_id	id_feedback → pedido_id; pedido_id → cliente_id (em PEDIDO)	FEEDBACK guardava cliente_id redundante; possibilitava inconsistência (feedback de cliente diferente do pedido)	CORRIGIDO no SQL: cliente_id removido de FEEDBACK. Cliente obtido via JOIN FEEDBACK→PEDIDO→CLIENTE (RN16 mantida)
Total de PEDIDO e ORCAMENTO	total = soma dos itens × quantidade (RN08)	Total é dado derivado; repetição de propósito para desempenho	Mantido e documentado como repetição intencional; conferido por consulta de auditoria (seção 11.5)
ORCAMENTO: cliente_id	id_orcamento → cliente_id e pedido_id (quando preenchido)	O cliente do orçamento poderia ser descoberto pelo pedido quando pedido_id está preenchido; mas pode ser NULL	Mantido: o orçamento pode estar aberto sem pedido, então cliente_id é necessário. Auditoria verifica coerência quando pedido existe

AGENDAMENTO: tipo_procedimento	id_agendamento → tipo_procedimento; hoje nenhum dado depende do tipo	Se serviços ganharem preço próprio, tipo_procedimento seria uma chave para outra tabela	Atendida na versão atual; se serviços ganharem preço próprio, criaremos tabela SERVICO (seção 8.5)

10.5 Resumo por tabela

Tabela	Chave primária e colunas únicas	O que a chave define	Forma normal
cliente	id_cliente; email	id_cliente → nome, telefone, email, endereco, data_cadastro	3FN
categoria	id_categoria; nome	id_categoria → nome, descricao	3FN
fornecedor	id_fornecedor; cnpj	id_fornecedor → nome, cnpj, telefone, email, endereco	3FN
pet	id_pet	id_pet → nome, tipo, raca, idade, cliente_id	3FN
agendamento	id_agendamento	id_agendamento → data, horario, tipo_procedimento, status, pet_id	3FN
produto	id_produto	id_produto → nome, descricao, preco, categoria_id	3FN
estoque	id_estoque; produto_id	id_estoque → quantidade, estoque_minimo, produto_id	3FN
fornecedor_produto	id; (fornecedor_id, produto_id)	fornecedor_id + produto_id → preco_compra, data_fornecimento	3FN
pedido	id_pedido	id_pedido → data, status, total, cliente_id	3FN (total repetido de propósito)
item_pedido	id_item	id_item → quantidade, preco_unitario, pedido_id, produto_id	3FN
orcamento	id_orcamento	id_orcamento → data, status, total, cliente_id, pedido_id	3FN (cliente_id necessário enquanto pedido_id é NULL)
feedback	id_feedback	id_feedback → nota, comentario, pedido_id	3FN (cliente_id removido)

11 SCRIPT SQL COMPLETO

Esta seção apresenta o script SQL completo do Pet House em MySQL. Ele inclui a criação do banco e das tabelas (CREATE), a inserção dos dados de teste (INSERT), exemplos de consulta com JOIN (SELECT), atualização de registros (UPDATE), exclusão (DELETE) e as consultas gerenciais e de auditoria.

11.1 Criando o banco de dados

```
CREATE DATABASE IF NOT EXISTS `pethouse` CHARACTER SET utf8mb4;
USE `pethouse`;
-- Conecte-se ao banco antes de executar o restante:
-- No phpMyAdmin basta importar. No terminal: USE pethouse;
```

11.2 Ordem de criação das tabelas

Nível	Tabelas	Por quê
1	cliente, categoria, fornecedor	Não dependem de nenhuma outra tabela
2	pet, produto	pet precisa de cliente; produto precisa de categoria
3	agendamento, estoque, fornecedor_produto, pedido	agendamento precisa de pet; estoque e fornecedor_produto precisam de produto; pedido precisa de cliente
4	item_pedido, orcamento, feedback	precisam de pedido e/ou produto

11.3 Script de criação das tabelas (CREATE)

```
-- ============================================================
-- Pet House - Script de criação (MySQL / phpMyAdmin)
-- ============================================================
BEGIN;

-- Tabelas sem dependências
CREATE TABLE cliente (
  id_cliente   INT NOT NULL AUTO_INCREMENT,
  nome         VARCHAR(120) NOT NULL,
  telefone     VARCHAR(20),
  email        VARCHAR(150) UNIQUE,
  endereco     VARCHAR(255),
  data_cadastro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE categoria (
  id_categoria INT NOT NULL AUTO_INCREMENT,
  nome         VARCHAR(100) NOT NULL UNIQUE,
  descricao    TEXT
);

CREATE TABLE fornecedor (
```

```
  id_fornecedor INT NOT NULL AUTO_INCREMENT,
  nome          VARCHAR(150) NOT NULL,
  cnpj          VARCHAR(18)  NOT NULL UNIQUE,
  telefone      VARCHAR(20),
  email         VARCHAR(150),
  endereco      VARCHAR(255)
);

-- Nível 2
CREATE TABLE pet (
  id_pet       INT NOT NULL AUTO_INCREMENT,
  nome         VARCHAR(100) NOT NULL,
  tipo         VARCHAR(50)  NOT NULL,
  raca         VARCHAR(100),
  idade        SMALLINT CHECK (idade >= 0),
  cliente_id   INTEGER NOT NULL,
  CONSTRAINT fk_pet_cliente FOREIGN KEY (cliente_id)
    REFERENCES cliente (id_cliente) ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE produto (
  id_produto   INT NOT NULL AUTO_INCREMENT,
  nome         VARCHAR(100) NOT NULL,
  descricao    VARCHAR(200),
  preco        DECIMAL(10,2) NOT NULL CHECK (preco > 0),
  categoria_id INTEGER NOT NULL,
  CONSTRAINT fk_produto_categoria FOREIGN KEY (categoria_id)
    REFERENCES categoria (id_categoria)
);

-- Nível 3
CREATE TABLE agendamento (
  id_agendamento   INT NOT NULL AUTO_INCREMENT,
  data             DATE        NOT NULL,
  horario          TIME        NOT NULL,
  tipo_procedimento VARCHAR(50) NOT NULL,
  status           VARCHAR(20) NOT NULL DEFAULT 'agendado'
    CHECK (status IN
('agendado','concluido','cancelado','nao_compareceu')),
  pet_id           INTEGER NOT NULL,
  CONSTRAINT fk_ag_pet FOREIGN KEY (pet_id) REFERENCES pet (id_pet)
);

CREATE TABLE estoque (
  id_estoque     INT NOT NULL AUTO_INCREMENT,
  quantidade     INTEGER NOT NULL CHECK (quantidade >= 0),
  estoque_minimo INTEGER NOT NULL CHECK (estoque_minimo >= 0),
  produto_id     INTEGER NOT NULL UNIQUE,
```

```
  CONSTRAINT fk_est_produto FOREIGN KEY (produto_id) REFERENCES produto
(id_produto)
);

CREATE TABLE fornecedor_produto (
  id               INT NOT NULL AUTO_INCREMENT,
  fornecedor_id    INTEGER NOT NULL,
  produto_id       INTEGER NOT NULL,
  preco_compra     DECIMAL(10,2) NOT NULL CHECK (preco_compra > 0),
  data_fornecimento DATE NOT NULL,
  CONSTRAINT fk_fp_fornecedor FOREIGN KEY (fornecedor_id) REFERENCES
fornecedor (id_fornecedor),
  CONSTRAINT fk_fp_produto    FOREIGN KEY (produto_id)    REFERENCES
produto (id_produto),
  CONSTRAINT uq_fp            UNIQUE (fornecedor_id, produto_id)
);

CREATE TABLE pedido (
  id_pedido  INT NOT NULL AUTO_INCREMENT,
  data       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status     VARCHAR(20) NOT NULL DEFAULT 'pendente'
    CHECK (status IN ('pendente','pago','entregue','cancelado')),
  total      DECIMAL(10,2) NOT NULL CHECK (total >= 0),
  cliente_id INTEGER NOT NULL,
  CONSTRAINT fk_ped_cliente FOREIGN KEY (cliente_id) REFERENCES cliente
(id_cliente)
);

-- Nível 4
CREATE TABLE item_pedido (
  id_item        INT NOT NULL AUTO_INCREMENT,
  quantidade     INTEGER       NOT NULL CHECK (quantidade > 0),
  preco_unitario DECIMAL(10,2) NOT NULL CHECK (preco_unitario > 0),
  pedido_id      INTEGER NOT NULL,
  produto_id     INTEGER NOT NULL,
  CONSTRAINT fk_item_pedido  FOREIGN KEY (pedido_id)  REFERENCES pedido
(id_pedido),
  CONSTRAINT fk_item_produto FOREIGN KEY (produto_id) REFERENCES
produto (id_produto)
);

CREATE TABLE orcamento (
  id_orcamento INT NOT NULL AUTO_INCREMENT,
  data         DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status       VARCHAR(20) NOT NULL DEFAULT 'aberto'
    CHECK (status IN ('aberto','aprovado','recusado')),
  total        DECIMAL(10,2) NOT NULL CHECK (total >= 0),
  cliente_id   INTEGER NOT NULL,
  pedido_id    INTEGER,  -- NULL enquanto orçamento não virou pedido
```

```
  CONSTRAINT fk_orc_cliente FOREIGN KEY (cliente_id) REFERENCES cliente
(id_cliente),
  CONSTRAINT fk_orc_pedido  FOREIGN KEY (pedido_id)  REFERENCES pedido
(id_pedido)
);

-- FEEDBACK: cliente_id REMOVIDO (3FN) - obtido via JOIN com PEDIDO
CREATE TABLE feedback (
  id_feedback INT NOT NULL AUTO_INCREMENT,
  nota        SMALLINT NOT NULL CHECK (nota BETWEEN 1 AND 5),
  comentario  VARCHAR(300),
  pedido_id   INTEGER NOT NULL,
  CONSTRAINT fk_fb_pedido FOREIGN KEY (pedido_id) REFERENCES pedido
(id_pedido)
);

COMMIT;
```

11.4 Consultas de leitura com SELECT e JOIN

As consultas abaixo demonstram o uso de SELECT com JOIN, WHERE, ORDER BY, GROUP BY e funções de agregação. Elas cobrem os casos de uso mais comuns do sistema (RF05, RF10, RF13, RF17, RF18).

```
-- Q01: Agenda do dia (todos os agendamentos de hoje)

SELECT a.data, a.horario, a.tipo_procedimento, a.status,
       p.nome AS pet, c.nome AS tutor, c.telefone
FROM agendamento a
JOIN pet p ON p.id_pet = a.pet_id
JOIN cliente c ON c.id_cliente = p.cliente_id
WHERE a.data = CURRENT_DATE
ORDER BY a.horario;
```

```
-- Q02: Histórico de pedidos de um cliente

SELECT ped.id_pedido, ped.data, ped.status, ped.total
FROM pedido ped
JOIN cliente c ON c.id_cliente = ped.cliente_id
WHERE c.email = 'ana.souza@email.com'
ORDER BY ped.data DESC;
```

```
-- Q03: Itens de um pedido com nome do produto

SELECT ip.id_item, pr.nome AS produto, ip.quantidade,
       ip.preco_unitario,
       (ip.quantidade * ip.preco_unitario) AS subtotal
FROM item_pedido ip
JOIN produto pr ON pr.id_produto = ip.produto_id
WHERE ip.pedido_id = 1;
```

```
-- Q04: Estoque abaixo do mínimo (alerta de reposição - RF13)

SELECT pr.nome, e.quantidade, e.estoque_minimo,
       (e.estoque_minimo - e.quantidade) AS falta
FROM estoque e
JOIN produto pr ON pr.id_produto = e.produto_id
WHERE e.quantidade < e.estoque_minimo
ORDER BY falta DESC;
```

```
-- Q05: Menor preço de compra por produto (comparativo de fornecedores)

SELECT pr.nome AS produto,
       f.nome  AS fornecedor_mais_barato,
       fp.preco_compra AS menor_preco
FROM fornecedor_produto fp
JOIN produto    pr ON pr.id_produto    = fp.produto_id
JOIN fornecedor f  ON f.id_fornecedor = fp.fornecedor_id
WHERE fp.preco_compra = (
  SELECT MIN(fp2.preco_compra)
  FROM fornecedor_produto fp2
  WHERE fp2.produto_id = fp.produto_id
)
ORDER BY pr.nome;
```

```
-- Q06: Feedback com dados do cliente (via JOIN - sem cliente_id em
feedback)

SELECT fb.id_feedback, fb.nota, fb.comentario,
       c.nome AS cliente, ped.data AS data_pedido
FROM feedback fb
JOIN pedido ped ON ped.id_pedido = fb.pedido_id
JOIN cliente c  ON c.id_cliente  = ped.cliente_id
ORDER BY fb.nota DESC;
```

11.4.1 Exemplos de UPDATE e DELETE

```
-- U01: Atualizar status de agendamento para concluído
UPDATE agendamento SET status = 'concluido'
WHERE id_agendamento = 4;

-- U02: Ajustar estoque após venda de 2 unidades do produto 1
UPDATE estoque SET quantidade = quantidade - 2
WHERE produto_id = 1;

-- U03: Alterar preço de venda de um produto
UPDATE produto SET preco = 179.90
WHERE id_produto = 1;
```

```
-- U04: Aprovar orçamento e vincular ao pedido gerado
UPDATE orcamento SET status = 'aprovado', pedido_id = 9
WHERE id_orcamento = 4;

-- D01: Excluir feedback (quando solicitado pelo cliente - LGPD)
DELETE FROM feedback WHERE id_feedback = 5;

-- D02: Cancelar agendamento
UPDATE agendamento SET status = 'cancelado'
WHERE id_agendamento = 9;
-- Nota: não deletar agendamentos; manter histórico com status
cancelado.
```

11.5 Consultas gerenciais e de auditoria

As consultas a seguir respondem a perguntas de negócio e verificam a integridade dos dados. Elas são essenciais para o uso operacional e gerencial do sistema.

11.5.1 Consultas gerenciais

```
-- G01: Vendas por período (faturamento mensal)
SELECT
  DATE_FORMAT(data, '%Y-%m') AS mes,
  COUNT(id_pedido) AS qtd_pedidos,
  SUM(total) AS faturamento
FROM pedido
WHERE status IN ('pago','entregue')
  AND data BETWEEN '2026-10-01' AND '2026-10-31'
GROUP BY DATE_FORMAT(data, '%Y-%m')
ORDER BY mes;
```

```
-- G02: Produtos mais vendidos (TOP 5 por quantidade)
SELECT pr.nome, SUM(ip.quantidade) AS total_vendido
FROM item_pedido ip
JOIN produto pr ON pr.id_produto = ip.produto_id
JOIN pedido ped ON ped.id_pedido = ip.pedido_id
WHERE ped.status IN ('pago','entregue')
GROUP BY pr.nome
ORDER BY total_vendido DESC
LIMIT 5;
```

```
-- G03: Clientes com maior gasto total (TOP 10)
SELECT c.nome, c.telefone,
       COUNT(ped.id_pedido) AS pedidos,
       SUM(ped.total) AS gasto_total
FROM cliente c
JOIN pedido ped ON ped.id_cliente = c.id_cliente
WHERE ped.status IN ('pago','entregue')
GROUP BY c.id_cliente, c.nome, c.telefone
```

```
ORDER BY gasto_total DESC
LIMIT 10;

-- G04: Média de avaliação por produto (via pedido e itens)
SELECT pr.nome AS produto, ROUND(AVG(fb.nota),2) AS media_nota,
       COUNT(fb.id_feedback) AS qtd_avaliacoes
FROM feedback fb
JOIN pedido ped ON ped.id_pedido = fb.pedido_id
JOIN item_pedido ip ON ip.pedido_id = ped.id_pedido
JOIN produto pr ON pr.id_produto = ip.produto_id
GROUP BY pr.nome
ORDER BY media_nota DESC;

-- G05: Agendamentos por tipo de procedimento no mês
SELECT tipo_procedimento,
       COUNT(*) AS total,
       SUM(CASE WHEN status = 'concluido' THEN 1 ELSE 0 END) AS
concluidos,
       SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) AS
cancelados
FROM agendamento
WHERE data BETWEEN '2026-10-01' AND '2026-10-31'
GROUP BY tipo_procedimento
ORDER BY total DESC;

-- G06: Margem de lucro bruta por produto (preço venda - menor preço
compra)
SELECT pr.nome,
       pr.preco AS preco_venda,
       MIN(fp.preco_compra) AS menor_custo,
       (pr.preco - MIN(fp.preco_compra)) AS margem_bruta,
       ROUND(((pr.preco - MIN(fp.preco_compra)) / pr.preco * 100),2) AS
margem_pct
FROM produto pr
JOIN fornecedor_produto fp ON fp.produto_id = pr.id_produto
GROUP BY pr.id_produto, pr.nome, pr.preco
ORDER BY margem_pct DESC;

-- G07: Pets mais frequentes (mais agendamentos)
SELECT p.nome AS pet, p.tipo, c.nome AS tutor,
       COUNT(a.id_agendamento) AS total_agendamentos
FROM pet p
JOIN agendamento a ON a.pet_id = p.id_pet
JOIN cliente c ON c.id_cliente = p.cliente_id
GROUP BY p.id_pet, p.nome, p.tipo, c.nome
ORDER BY total_agendamentos DESC
LIMIT 10;

-- G08: Orçamentos abertos há mais de 7 dias (pendentes de resposta)
```

```
SELECT o.id_orcamento, c.nome AS cliente, c.telefone,
       o.data, o.total,
       (CURRENT_DATE - o.data::date) AS dias_aberto
FROM orcamento o
JOIN cliente c ON c.id_cliente = o.cliente_id
WHERE o.status = 'aberto'
  AND o.data < NOW() - INTERVAL 7 DAY
ORDER BY dias_aberto DESC;
```

```
-- G09: Pedidos pendentes (a receber)
SELECT ped.id_pedido, c.nome, c.telefone,
       ped.data, ped.total
FROM pedido ped
JOIN cliente c ON c.id_cliente = ped.cliente_id
WHERE ped.status = 'pendente'
ORDER BY ped.data;
```

```
-- G10: Resumo de estoque por categoria
SELECT cat.nome AS categoria,
       COUNT(pr.id_produto) AS qtd_produtos,
       SUM(e.quantidade) AS estoque_total,
       SUM(CASE WHEN e.quantidade < e.estoque_minimo THEN 1 ELSE 0 END)
AS produtos_baixo_estoque
FROM categoria cat
JOIN produto pr ON pr.categoria_id = cat.id_categoria
JOIN estoque e  ON e.produto_id = pr.id_produto
GROUP BY cat.nome
ORDER BY produtos_baixo_estoque DESC;
```

11.5.2 Consultas de auditoria

As consultas de auditoria verificam a integridade dos dados e detectam possíveis inconsistências apontadas na análise de normalização.

```
-- A01: Total do pedido diverge da soma dos itens (anomalia de
integridade)
SELECT ped.id_pedido, ped.total AS total_gravado,
       COALESCE(SUM(ip.quantidade * ip.preco_unitario),0) AS
total_calculado,
       ABS(ped.total - COALESCE(SUM(ip.quantidade *
ip.preco_unitario),0)) AS diferenca
FROM pedido ped
LEFT JOIN item_pedido ip ON ip.pedido_id = ped.id_pedido
GROUP BY ped.id_pedido, ped.total
HAVING ABS(ped.total - COALESCE(SUM(ip.quantidade *
ip.preco_unitario),0)) > 0.01;
```

```
-- A02: Orçamentos aprovados sem pedido vinculado (inconsistência de
fluxo)
SELECT o.id_orcamento, c.nome AS cliente, o.data, o.total
```

```
FROM orcamento o
JOIN cliente c ON c.id_cliente = o.cliente_id
WHERE o.status = 'aprovado'
  AND o.pedido_id IS NULL;

-- A03: Pedidos sem nenhum item (pedido vazio - viola RN05)
SELECT ped.id_pedido, c.nome AS cliente, ped.data, ped.total
FROM pedido ped
JOIN cliente c ON c.id_cliente = ped.cliente_id
LEFT JOIN item_pedido ip ON ip.pedido_id = ped.id_pedido
WHERE ip.id_item IS NULL
  AND ped.status != 'cancelado';

-- A04: Estoque com fornecedor_id inconsistente com FORNECEDOR_PRODUTO
-- (verifica que o fornecedor_id no estoque realmente fornece o
produto)
SELECT e.id_estoque, pr.nome AS produto,
       e.quantidade, e.estoque_minimo
FROM estoque e
JOIN produto pr ON pr.id_produto = e.produto_id
-- Alerta: ESTOQUE não tem mais fornecedor_id após normalização.
-- Para saber o fornecedor ideal, use FORNECEDOR_PRODUTO.
-- Esta consulta lista produtos com estoque abaixo do mínimo
-- e seus fornecedores cadastrados:
SELECT e.produto_id, pr.nome, e.quantidade, e.estoque_minimo,
       f.nome AS fornecedor_sugerido, fp.preco_compra
FROM estoque e
JOIN produto pr ON pr.id_produto = e.produto_id
JOIN fornecedor_produto fp ON fp.produto_id = e.produto_id
JOIN fornecedor f ON f.id_fornecedor = fp.fornecedor_id
WHERE e.quantidade < e.estoque_minimo
ORDER BY pr.nome, fp.preco_compra;

-- A05: Clientes duplicados por telefone (possível duplicidade de
cadastro)
SELECT telefone, COUNT(*) AS ocorrencias,
       GROUP_CONCAT(nome SEPARATOR ' / ') AS nomes
FROM cliente
GROUP BY telefone
HAVING COUNT(*) > 1
ORDER BY ocorrencias DESC;
```

12 DADOS FICTÍCIOS PARA TESTE

Os dados abaixo foram criados para simular o uso real do sistema. O script deve ser executado após o script da seção 11.3. Todos os totais de pedido foram conferidos contra a soma dos itens.

Os dados foram elaborados com base nos requisitos e regras de negócio com base nos requisitos e regras de negócio definidos nas seções 5 e 6. Não foi utilizada inteligência artificial para a geração dos registros.

Total de registros inseridos: 5 categorias, 4 fornecedores, 8 clientes, 12 pets, 12 agendamentos, 15 produtos, 15 registros de estoque, 16 registros de fornecedor_produto, 8 pedidos, 16 itens de pedido, 5 orçamentos, 5 feedbacks. Total: 121 registros.

```
-- Rode DEPOIS do schema_pet_house.sql
BEGIN;

-- CATEGORIA
INSERT INTO categoria (id_categoria, nome, descricao) VALUES
  (1, 'Alimentacao',      'Racoes, saches e petiscos para caes e
gatos'),
  (2, 'Higiene e Beleza', 'Shampoos, condicionadores e escovas'),
  (3, 'Brinquedos',       'Brinquedos e arranhadores'),
  (4, 'Acessorios',       'Coleiras, peitorais, camas e guias'),
  (5, 'Saude e Bem-estar','Vermifugos, antipulgas e suplementos');

-- FORNECEDOR
INSERT INTO fornecedor (id_fornecedor, nome, cnpj, telefone, email,
endereco) VALUES
  (1,'NutriPet Alimentos Ltda','12.345.678/0001-90','(11) 3456-
7890','contato@nutripet.com.br','Av. das Industrias, 1500 - Sao
Paulo/SP'),
  (2,'BelaPet Cosmeticos','23.456.789/0001-01','(11) 3567-
8901','vendas@belapet.com.br','Rua das Flores, 220 - Guarulhos/SP'),
  (3,'Diverte Pet Acessorios','34.567.890/0001-12','(19) 3678-
9012','comercial@divertepet.com','Rod. Anhanguera km 98 -
Campinas/SP'),
  (4,'VetSaude Distribuidora','45.678.901/0001-23','(11) 3789-
0123','pedidos@vetsaude.com.br','Rua Dr. Almeida, 85 - Sao Paulo/SP');

-- CLIENTE
INSERT INTO cliente (id_cliente, nome, telefone, email, endereco,
data_cadastro) VALUES
  (1,'Ana Beatriz Souza','(11) 98765-4321','ana.souza@email.com','Rua
das Acacias, 120 - Vila Mariana, Sao Paulo/SP','2026-01-15 10:30:00'),
  (2,'Carlos Eduardo Lima','(11) 97654-
3210','carlos.lima@email.com','Av. Paulista, 1578 - Bela Vista, Sao
Paulo/SP','2026-02-03 14:10:00'),
  (3,'Fernanda Oliveira','(11) 96543-
2109','fernanda.oliveira@email.com','Rua Augusta, 900 - Consolacao, Sao
Paulo/SP','2026-02-20 09:45:00'),
  (4,'Ricardo Mendes','(11) 95432-1098','ricardo.mendes@email.com','Rua
Oscar Freire, 455 - Pinheiros, Sao Paulo/SP','2026-03-11 16:00:00'),
```

```
  (5,'Juliana Castro','(11) 94321-0987','juliana.castro@email.com','Rua
Vergueiro, 2300 - Liberdade, Sao Paulo/SP','2026-04-07 11:20:00'),
  (6,'Marcos Vinicius Rocha','(11) 93210-
9876','marcos.rocha@email.com','Rua Domingos de Morais, 640 - Vila
Mariana, Sao Paulo/SP','2026-05-18 13:35:00'),
  (7,'Patricia Almeida','(11) 92109-
8765','patricia.almeida@email.com','Av. Brig. Faria Lima, 3000 - Itaim
Bibi, Sao Paulo/SP','2026-06-25 15:50:00'),
  (8,'Lucas Ferreira','(11) 91098-7654','lucas.ferreira@email.com','Rua
Teodoro Sampaio, 780 - Pinheiros, Sao Paulo/SP','2026-08-02 08:15:00');

-- PET
INSERT INTO pet (id_pet, nome, tipo, raca, idade, cliente_id) VALUES
  (1,'Thor','Cao','Golden Retriever',4,1),
(2,'Mel','Gato','Siames',2,1),
  (3,'Bobby','Cao','Poodle',7,2),(4,'Luna','Gato','Persa',3,3),
  (5,'Rex','Cao','Pastor Alemao',5,4),(6,'Nina','Cao','Shih Tzu',2,5),
  (7,'Pipoca','Coelho','Mini Lop',1,5),(8,'Zeca','Cao','Vira-
lata',9,6),
  (9,'Mimi','Gato','SRD',6,7),(10,'Bidu','Cao','Beagle',3,7),
  (11,'Kiwi','Ave','Calopsita',2,8),(12,'Fred','Cao','Bulldog
Frances',4,8);

-- PRODUTO
INSERT INTO produto (id_produto, nome, descricao, preco, categoria_id)
VALUES
  (1,'Racao Premium Caes Adultos 15kg','Racao super premium sabor
frango e arroz',189.90,1),
  (2,'Racao Premium Gatos Castrados 10kg','Racao com controle de peso
para gatos',159.90,1),
  (3,'Petisco Bifinho Carne 500g','Petisco macio sabor carne para
caes',24.90,1),
  (4,'Sache Gatos Filhotes 85g','Alimento umido para gatos
filhotes',4.50,1),
  (5,'Shampoo Neutro 500ml','Shampoo hipoalergenico para caes e
gatos',32.00,2),
  (6,'Condicionador Pelos Longos 500ml','Facilita o desembaraco e deixa
o pelo macio',35.50,2),
  (7,'Escova Removedora de Pelos','Escova com cerdas macias para
remocao de pelos',29.90,2),
  (8,'Bola de Borracha Macica','Brinquedo resistente para caes de medio
porte',18.90,3),
  (9,'Arranhador Torre para Gatos','Torre com 3 andares e
esconderijo',119.00,3),
  (10,'Corda Mordedor','Corda de algodao trancado',15.50,3),
  (11,'Coleira Ajustavel M','Coleira de nylon com fecho de
seguranca',39.90,4),
  (12,'Cama Pet Media','Cama acolchoada lavavel para porte
medio',89.90,4),
  (13,'Peitoral com Guia','Peitoral acolchoado com guia de
1,2m',54.90,4),
```

```
  (14,'Vermifugo Caes 4 comprimidos','Vermifugo de amplo espectro para caes',49.90,5),
  (15,'Antipulgas Gatos Pipeta','Pipeta antipulgas e carrapatos para gatos',69.90,5);

-- ESTOQUE (1 registro por produto)
INSERT INTO estoque (id_estoque, quantidade, estoque_minimo, produto_id) VALUES
  (1,25,10,1),(2,18,8,2),(3,60,20,3),(4,150,50,4),(5,40,15,5),
  (6,35,15,6),(7,22,10,7),(8,45,15,8),(9,8,5,9),(10,70,25,10),
  (11,30,12,11),(12,12,6,12),(13,20,8,13),(14,28,10,14),
  (15,4,10,15);  -- produto 15 abaixo do mínimo (bom para testar alertas)

-- FORNECEDOR_PRODUTO
INSERT INTO fornecedor_produto (id, fornecedor_id, produto_id, preco_compra, data_fornecimento) VALUES
  (1,1,1,135.00,'2026-09-10'),(2,1,2,112.00,'2026-09-10'),
(3,1,3,14.50,'2026-09-10'),(4,1,4,2.80,'2026-09-10'),
  (5,2,5,19.00,'2026-09-15'),(6,2,6,21.50,'2026-09-15'),
(7,2,7,16.00,'2026-09-15'),
  (8,3,7,17.50,'2026-09-20'),(9,3,8,9.50,'2026-09-20'),
(10,3,9,75.00,'2026-09-20'),
  (11,3,10,8.00,'2026-09-20'),(12,3,11,22.00,'2026-09-22'),
(13,3,12,55.00,'2026-09-22'),
  (14,3,13,32.00,'2026-09-22'),(15,4,14,31.00,'2026-10-01'),
(16,4,15,45.00,'2026-10-01');

-- PEDIDO
INSERT INTO pedido (id_pedido, data, status, total, cliente_id) VALUES
  (1,'2026-10-02 10:15:00','entregue',239.70,1),
  (2,'2026-10-05 15:40:00','entregue',186.90,2),
  (3,'2026-10-10 11:05:00','pago',97.40,3),
  (4,'2026-10-14 17:30:00','pago',150.00,4),
  (5,'2026-10-16 09:20:00','entregue',94.80,5),
  (6,'2026-10-22 13:55:00','pendente',189.70,6),
  (7,'2026-10-24 16:10:00','entregue',146.60,1),
  (8,'2026-10-25 12:00:00','cancelado',189.90,7);

-- ITEM_PEDIDO (totais conferem com a soma dos itens)
INSERT INTO item_pedido (id_item, quantidade, preco_unitario, pedido_id, produto_id) VALUES
  -- Pedido 1: 189.90 + 2x24.90 = 239.70
  (1,1,189.90,1,1),(2,2,24.90,1,3),
  -- Pedido 2: 159.90 + 6x4.50 = 186.90
  (3,1,159.90,2,2),(4,6,4.50,2,4),
  -- Pedido 3: 32.00 + 35.50 + 29.90 = 97.40
  (5,1,32.00,3,5),(6,1,35.50,3,6),(7,1,29.90,3,7),
  -- Pedido 4: 119.00 + 2x15.50 = 150.00
  (8,1,119.00,4,9),(9,2,15.50,4,10),
```

```
  -- Pedido 5: 39.90 + 54.90 = 94.80
  (10,1,39.90,5,11),(11,1,54.90,5,13),
  -- Pedido 6: 49.90 + 2x69.90 = 189.70
  (12,1,49.90,6,14),(13,2,69.90,6,15),
  -- Pedido 7: 89.90 + 3x18.90 = 146.60
  (14,1,89.90,7,12),(15,3,18.90,7,8),
  -- Pedido 8: 189.90
  (16,1,189.90,8,1);

-- ORCAMENTO
INSERT INTO orcamento (id_orcamento, data, status, total, cliente_id,
pedido_id) VALUES
  (1,'2026-10-01 18:00:00','aprovado',239.70,1,1),
  (2,'2026-10-09 10:30:00','aprovado',97.40,3,3),
  (3,'2026-10-15 14:20:00','aprovado',94.80,5,5),
  (4,'2026-10-20 09:10:00','aberto',320.00,8,NULL),
  (5,'2026-10-21 16:45:00','recusado',85.00,2,NULL);

-- FEEDBACK (cliente_id REMOVIDO - obtido via JOIN)
INSERT INTO feedback (id_feedback, nota, comentario, pedido_id) VALUES
  (1,5,'Entrega rapida e produtos de otima qualidade. Meu Thor adorou a
racao!',1),
  (2,4,'Gostei muito, so demorou um pouco para chegar.',2),
  (3,5,'Atendimento excelente e a coleira e muito confortavel.',5),
  (4,4,'A cama e otima, as bolinhas poderiam ser um pouco maiores.',7),
  (5,3,'Produtos bons, mas o pedido veio sem a nota fiscal.',3);

-- Ajusta sequences (necessário quando IDs são informados manualmente)
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('cliente','id_cliente'),(SELECT MAX(id_cliente) FROM cliente));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('categoria','id_categoria'),(SELECT MAX(id_categoria) FROM
categoria));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('fornecedor','id_fornecedor'),(SELECT MAX(id_fornecedor) FROM
fornecedor));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('pet','id_pet'),(SELECT MAX(id_pet) FROM pet));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('agendamento','id_agendamento'),(SELECT MAX(id_agendamento)
FROM agendamento));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('produto','id_produto'),(SELECT MAX(id_produto) FROM produto));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('estoque','id_estoque'),(SELECT MAX(id_estoque) FROM estoque));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('fornecedor_produto','id'),(SELECT MAX(id) FROM
fornecedor_produto));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('pedido','id_pedido'),(SELECT MAX(id_pedido) FROM pedido));
```

```
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('item_pedido','id_item'),(SELECT MAX(id_item) FROM
item_pedido));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('orcamento','id_orcamento'),(SELECT MAX(id_orcamento) FROM
orcamento));
-- [MySQL gerencia AUTO_INCREMENT automaticamente, não precisa de
setval]('feedback','id_feedback'),(SELECT MAX(id_feedback) FROM
feedback));
COMMIT;
```

13 CONCLUSÃO

A documentação apresentou o contexto que justifica o DER do Pet House: um pet shop que realizava seus processos de forma manual e que passa a contar com um ERP integrado a um banco de dados MySQL. Cada entidade do modelo responde a uma necessidade identificada, e cada relacionamento foi explicado por regras de negócio e por cardinalidades justificadas nos dois sentidos.

Nesta segunda entrega corrigimos tudo que o professor apontou: adicionamos o modelo lógico (seção 8A), classificamos cada entidade como forte, fraca ou associativa (seção 8.2), removemos o cliente_id redundante de FEEDBACK para atender à 3FN (seção 10), ampliamos o script SQL com SELECT, JOIN, UPDATE e DELETE (seção 11), escrevemos as consultas gerenciais e de auditoria (seção 11.5) e incluímos os indicadores de BI e as possibilidades de IA (seção 13A). O banco também foi migrado de PostgreSQL para MySQL, conforme o pedido desta entrega.

Como próximos passos, o sistema pode evoluir para incluir cadastro de funcionários, uma tabela própria de serviços com preço fixo e o controle financeiro detalhado. A criação de views e triggers no banco e o desenvolvimento da interface do ERP também são evoluções naturais do projeto.

13A. Indicadores de BI e Possibilidades de IA

Esta seção apresenta os principais indicadores de Business Intelligence (BI) que o banco de dados do Pet House permite calcular, e as possibilidades de uso de Inteligência Artificial (IA) para agregar valor ao negócio.

13A.1 Indicadores de BI

BI (Business Intelligence) é usar os dados que o sistema registra para gerar informações úteis para quem gerencia o negócio. Os indicadores abaixo já podem ser obtidos com as consultas da seção 11.5, sem nenhum sistema extra.

Indicador	Pergunta de negócio	Tabelas envolvidas	Consulta
Faturamento mensal	Quanto o pet shop faturou em cada mês?	pedido	G01
Ticket médio	Qual o valor médio dos pedidos?	pedido	SUM(total)/COUNT(*)
Produtos mais vendidos	Quais produtos vendem mais em quantidade?	item_pedido, produto	G02
Clientes mais valiosos	Quais clientes gastam mais (LTV)?	pedido, cliente	G03
Nota média por produto	Como os clientes avaliam cada produto?	feedback, item_pedido, produto	G04
Taxa de conversão de orçamentos	De cada 10 orçamentos, quantos viraram pedido?	orcamento	COUNT aprovados/COUNT total
Taxa de cancelamento de agendamentos	Quantos agendamentos são cancelados?	agendamento	G05
Estoque crítico	Quais produtos precisam ser repostos agora?	estoque, produto	Q04
Margem de lucro bruta	Qual a margem de cada produto (venda vs compra)?	produto, fornecedor_produto	G06
Frequência de visitas	Quais pets frequentam mais o pet shop?	agendamento, pet	G07

13A.2 Possibilidades de uso de IA

Os dados armazenados no banco do Pet House abrem diversas possibilidades de aplicação de Inteligência Artificial, que podem ser implementadas em fases futuras do projeto.

Aplicação de IA	Como funciona	Dados usados	Benefício
Recomendação de produtos	Analisar o histórico de compras de um cliente e dos clientes com perfil similar para sugerir produtos	cliente, pedido, item_pedido, produto	Aumento do ticket médio e da satisfação
Previsão de demanda	Modelo de séries temporais (ex.: Prophet) para prever quantos produtos serão vendidos no próximo mês	item_pedido, produto, data do pedido	Reposição de estoque na hora certa, reduzindo falta e excesso
Segmentação de clientes (clustering)	Agrupar clientes por comportamento de compra: frequência, valor gasto, variedade de produtos	cliente, pedido, item_pedido	Campanhas de marketing direcionadas por perfil
Análise de sentimento dos feedbacks	Processar o texto dos comentários com NLP para identificar pontos positivos e negativos	feedback.comentario	Identificar problemas de entrega, qualidade ou atendimento automaticamente
Previsão de churn	Identificar clientes que estão comprando menos ou parando de agendar	cliente, pedido, agendamento	Ação proativa de retenção antes de perder o cliente
Chatbot de atendimento	Usar os dados de agendamento e pedidos para responder perguntas automaticamente por WhatsApp	agendamento, pedido, cliente	Redução do trabalho manual de atendimento

Repositório do projeto

O código-fonte e os arquivos do projeto PetHouse estão disponíveis no GitHub:
Links diretos dos modelos do projeto:
Diagrama Entidade-Relacionamento (DER):
https://github.com/ruan65678-lang/PetHouse/blob/main/DER_PetHouse.png
Projeto Lógico do Banco de Dados:
https://github.com/ruan65678-lang/PetHouse/blob/main/Projeto_Logico_PetHouse.pdf
https://github.com/ruan65678-lang/PetHouse